Azure AI Foundry & Azure OpenAI — Secure Enterprise Deployment

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 6

Outcome . . . 6

Scope boundary . . . 6

1. Solution Overview — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 6

What this platform is . . . 6

How the key pieces work . . . 7

What you can control . . . 7

Scope — what's in . . . 8

Scope — what's out (be honest with buyers) . . . 8

Authentication and connections used . . . 8

The standard deployment model . . . 9

2. Requirements — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 9

R1. Azure OpenAI (Foundry) resource access approved . . . 9

R2. An existing VNet, subnet, and Private DNS capability for the AI resources . . . 9

R3. Custom subdomain enabled on the Azure OpenAI resource . . . 9

R4. Data-plane RBAC roles available and assignable to a managed identity . . . 10

R5. Region quota (TPM) headroom for your intended model deployments . . . 10

R6. Log Analytics workspace for diagnostics (and a cost budget owner) . . . 10

R7. Content-filter (Responsible AI) position for the deployments . . . 10

R8. Python 3 with current Azure AI SDKs, and az/Az for the helpers . . . 11

Global Requirements — AI (category-level) 11

G1. Azure subscription with AI service access and rights . . . 11

G2. Entra identity, managed identity, and RBAC . . . 11

G3. Private networking capability . . . 12

G4. IaC + Python tooling and source control . . . 12

G5. CI/CD platform . . . 12

G6. Data source and classification (for data-grounded solutions) . . . 12

G7. Content-safety and responsible-AI posture . . . 12

G8. Cost controls and quotas . . . 12

3. Implementation — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 13

Naming you will create (reuse these exact names) . . . 13

Step 1 — Assess: inventory the current AI estate (read-only) . . . 14

Step 2 — Configure: scaffold the Bicep repo . . . 14
Step 3 — Configure: deploy the private, keyless Azure OpenAI resource (non-prod) . . . 15
Step 4 — Configure: wire the private endpoint and DNS . . . 15
Step 5 — Configure: assign least-privilege data-plane RBAC to the managed identity . . . 15
Step 6 — Configure: deploy the model with an explicit quota and content filter . . . 16
Step 7 — Configure: turn on diagnostics and the budget . . . 16
Step 8 — Pilot: validate keyless access and isolation end to end . . . 16
Step 9 — Wire the CI/CD pipeline (what-if → approval → deploy) . . . 17
Step 10 — Expand: deploy to production, then onboard more projects/models . . . 17
5. Testing — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 17
Test 1 — Public network access is disabled . . . 17
Test 2 — Keys are disabled (keyless-only) . . . 18
Test 3 — Private endpoint resolves and the private path works . . . 18
Test 4 — Custom subdomain is set . . . 18
Test 5 — Least-privilege data-plane RBAC is assigned . . . 18
Test 6 — Model deployment has an explicit quota and a content filter . . . 18
Test 7 — The content filter actually blocks . . . 19
Test 8 — Diagnostics flow and the budget exists . . . 19
Test 9 — Deployment stack tracks and can roll back . . . 19
Test 10 — Pipeline enforces what-if and approval . . . 19
Exit criteria . . . 19
6. Troubleshooting — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 20
Access, resource creation, and region . . . 20
Custom subdomain (the keyless + private-endpoint prerequisite) . . . 20
Keyless / RBAC (401 / 403) . . . 20
Private endpoint / DNS / networking . . . 20
Model deployment / content filter . . . 21
Monitoring / cost . . . 21
Deployment stacks / IaC . . . 21
Pipeline . . . 22
7. Best Practices — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment 22
Keep it clean . . . 22
Decisions that belong to the customer . . . 23
Where to go next . . . 23
8. Deliverables Summary — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment
Documentation . . . 24

Automation artifacts 24
What the buyer receives 25
Outcome 25
Appendix A — PowerShell Scripts 26

Executive Summary — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

White-label · AI

Most teams stand up Azure OpenAI the fast way: create the resource in the portal, leave the **public endpoint on**, copy the **account key** into an app setting, and start calling models — with **no quota ceiling** and **no cost alert**. It demos in an afternoon and works until it doesn't. That key is a portable, long-lived credential that can be leaked or committed; the public endpoint is a data-exfiltration surface reachable from anywhere; and an unbounded token meter turns a runaway loop or a launch spike into a five-figure bill. This solution deploys **Azure AI Foundry** (hub + project) and an **Azure OpenAI** resource the enterprise way — **private-by-default, keyless, quota-managed, and monitored** — as versioned infrastructure-as-code, so the foundation your teams build custom AI apps on is governed from day one.

This is a complete, **deploy-it-today** package written for an administrator who has used the Azure portal but has never authored Bicep, disabled a Cognitive Services public endpoint, wired a private endpoint and private DNS, or made an Azure OpenAI call with a **managed identity instead of a key**.

What you get

• A plain-language **Solution Overview** of the platform — what an AI Foundry hub and project are, how the Azure OpenAI resource, private endpoints, managed-identity (keyless) auth, model deployments/quota, and content filtering fit together, and how `what-if`-first deployment keeps it safe.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate **Global Requirements — AI**).

• An **Implementation Guide** of precise, ordered steps — discover the current estate, deploy the hub/project + Azure OpenAI as code with **public access disabled** and **local-auth (keys) disabled**, wire private endpoints and DNS, assign **least-privilege data-plane RBAC** to a managed identity, deploy a model with an explicit **TPM quota** and a **content filter**, turn on **diagnostics and a budget**, then pilot before you expand.

• Five automation artifacts: a read-only **PowerShell discovery** script, the core `AIFOUNDRY-main.bicep` infrastructure template, a **Python** posture-validation script (azure SDK, keyless), a **PowerShell** cost/usage/posture report, and a **Python** keyless smoke-test that proves managed-identity access and a live model call end to end.

• **Testing** and **Troubleshooting** guides for a confident, pilot-first rollout.

What this delivers

An **Azure AI Foundry hub and project** plus an **Azure OpenAI** resource deployed from Bicep, with `publicNetworkAccess` **= Disabled** and a **private endpoint** on the organization's VNet resolving through **private DNS**; `disableLocalAuth` **= true** so account keys cannot be used at all; a **user-assigned managed identity** holding the least-privilege **Cognitive Services OpenAI User** data-plane role; a **model deployment** with an explicit **tokens-per-minute (TPM) quota** and rate limit; a **content-filter (RAI) policy** in front of generation; **diagnostic logging** to Log Analytics; and a **budget with cost alerts** — a private, keyless, quota-managed, monitored foundation, proven in a non-production project before it reaches production.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — every deployment is preview-first (`what-if`), reviewed in a pull request, and approval-gated, with a documented rollback via deployment-stack delete/update. Content-acceptability, acceptable-AI-use, and data-classification decisions belong to your legal, trust-and-safety, and compliance functions; this package delivers the technical controls, not those determinations.

Outcome

A secure, governed **Azure OpenAI + AI Foundry** platform expressed as code in Git — private, keyless, quota-managed, and monitored — that your developers build enterprise AI applications on with confidence, deployable by a single administrator and validated before you scale.

Scope boundary

This solution **builds and operates custom AI** on Azure OpenAI / AI Foundry. It is **not** Microsoft 365 Copilot governance — that is covered by the **Copilot** and **Purview AI** catalog areas. Where those govern a Microsoft-hosted assistant over your M365 data, this gives your teams a private, controlled Azure OpenAI platform to build their own AI apps on. The two are complementary; cross-reference, don't duplicate.

1. Solution Overview — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

What this platform is

This solution stands up the **foundation your teams build custom AI applications on** — an **Azure AI Foundry** hub and project, and an **Azure OpenAI** resource — configured the enterprise way from the first deploy: **private networking**, **no keys**, **managed model quota**, **content filtering**, and **monitoring**, all expressed as **infrastructure-as-code** (Bicep) in Git rather than clicked together in the portal.

Two things are worth separating up front, because people conflate them:

- **Azure OpenAI** is the *service* that hosts the models (GPT-family chat/completions, embeddings, and others) behind an Azure endpoint you call from your apps. It is a **Cognitive Services** account of kind `OpenAI`.
- **Azure AI Foundry** (the platform formerly surfaced as *Azure AI Studio*) is the *workspace* where teams build, test, and manage AI projects — a **hub** provides shared security, networking, and connections, and one or more **projects** under it are where individual teams work and connect to model deployments. *The Foundry resource model has evolved (hub/project workspaces, and a newer unified Foundry resource) — verify the current resource types and portal layout in your tenant.*

The point of doing this as a governed platform, not a quick portal resource, is that **custom AI touches sensitive data and can emit harmful or ungrounded output**. So the platform is **private-by-default** (nothing reachable from the public internet), **keyless** (identities, not shared secrets, authenticate), **quota-bounded** (a token ceiling per model so spend and load are predictable), and **guarded and**

observed (a content filter in front of generation, full diagnostic logs behind it).

How the key pieces work

• **Private networking.** By default an Azure OpenAI resource has a **public endpoint** reachable from anywhere. Setting `publicNetworkAccess` **= Disabled** turns that off, and a **private endpoint** gives the resource a private IP inside your VNet. A **private DNS zone** (for example `privatelink.openai.azure.com` — *verify the exact zone name(s) for OpenAI / AI Services in your cloud*) makes the resource's hostname resolve to that private IP, so your apps reach it entirely over the Microsoft backbone. Reaching Azure OpenAI over the public internet becomes impossible.

• **Keyless (managed identity + RBAC) auth.** Every Cognitive Services account is created with two **account keys** — long-lived, portable secrets that authenticate as the whole resource. Setting `disableLocalAuth` **= true** switches them off entirely. Your app then authenticates with a **managed identity**: Azure issues it an Entra token, and the token is authorized by an Azure **role** on the resource — the least-privilege data-plane role is **Cognitive Services OpenAI User** (call models) or **Cognitive Services OpenAI Contributor** (also manage deployments). Nothing to leak, rotate, or commit. Keyless auth requires the resource to have a **custom subdomain** (`customSubDomainName`).

• **Model deployments, versioning, and quota.** You do not call "GPT-4o" directly — you create a **deployment** (a named instance of a specific model **version**) on the resource, and your app calls the *deployment name*. Each deployment carries a **capacity** measured in **tokens-per-minute (TPM)** drawn from your regional **quota**; TPM also sets the requests-per-minute rate limit. This is your primary lever against both **load** and **bill shock** — a deployment cannot burn more than the TPM you granted it. *Model and version availability, regional availability, and quota defaults vary and change — verify current availability in your subscription and region.*

• **Content filtering (Responsible AI).** Azure OpenAI applies a **content filter** to prompts and completions across hate, sexual, violence, and self-harm categories. You attach a **RAI (content-filter) policy** to each deployment and set severity thresholds; additional protections (prompt-shield / jailbreak detection, protected-material detection, asynchronous filtering) exist and **some are in preview** — *verify current availability and preview status*. The **acceptable-content threshold is your trust-and-safety decision**; the platform enforces whatever you choose.

• **Monitoring and cost.** A **diagnostic setting** streams the resource's audit, request/response metadata, and metrics to a **Log Analytics** workspace, so every call is observable. A **budget** with **cost alerts** on the resource group warns you long before an overrun, and per-deployment TPM caps the theoretical maximum spend.

• **Key Vault for any unavoidable secret.** Keyless is the default. If a specific downstream connection genuinely needs a secret (a third-party API key a Foundry connection uses, say), it goes in **Key Vault** referenced by the managed identity — never in code or app settings.

What you can control

• **Isolation** — public access disabled, private endpoint(s) and DNS on your chosen VNet/subnet.

• **Identity** — which managed identity/identities authenticate, and at which least-privilege role.

• **Models** — which models/versions are deployed, their deployment names, and their **TPM quota** and rate limits.

- **Safety** — the content-filter policy attached to each deployment and its severity thresholds.
- **Observability & spend** — the Log Analytics destination, the budget amount, and the alert thresholds.
- **Rollout scope** — a non-production Foundry project first, production only after validation and approval.

Scope — what's in

- An **Azure AI Foundry hub + project** and an **Azure OpenAI** resource, deployed from Bicep.
- **Private endpoint(s)**, private DNS, and **disabled public access** on the AI resources.
- **Keyless** auth: `disableLocalAuth`, a user-assigned **managed identity**, and least-privilege data-plane RBAC; **Key Vault** for any unavoidable secret.
- One or more **model deployments** with explicit **TPM quota**/rate limits and a **content-filter policy**.
- **Diagnostic logging** to Log Analytics and a **budget** with cost alerts.

Scope — what's out (be honest with buyers)

- **This is not Microsoft 365 Copilot governance.** Governing Copilot over your M365 data is the **Copilot** and **Purview AI** catalog areas; this is a platform for building your *own* AI apps. Cross-referenced, not duplicated.
- **Not a RAG / grounding build.** Connecting the platform to your private data with Azure AI Search and permission-trimming is the sibling **Enterprise RAG Platform** solution.
- **Not the enterprise landing zone.** Management groups, org-wide policy, and hub-spoke backbone are the **AI Landing Zone** / DevOps landing-zone solutions; this deploys the AI platform *into* an existing VNet.
- **Not the application.** This is the governed model platform; the app that consumes it (prompt design, orchestration, UX) is yours to build on top.
- **No content-policy or acceptable-use determination.** The platform enforces the thresholds you set; *what* is acceptable is your legal / trust-and-safety decision.

Authentication and connections used

- **Azure CLI / Az PowerShell** signed in as an identity with **Contributor/Owner** at the deployment scope, for local `what-if`, discovery, and the deploy helper (per Global Requirements G1).
- A **user-assigned managed identity** that your application runs as, holding the **Cognitive Services OpenAI User** data-plane role — the keyless path apps use at runtime (G2).
- A **CI/CD service connection** to Azure using **workload identity federation (OIDC)** — no stored secret — for the pipeline (G5).
- **Private endpoints** on your existing **VNet/subnet** with **private DNS** (G3); the Python scripts authenticate via `DefaultAzureCredential`, so they use your signed-in identity or the managed identity with **no key**.

The standard deployment model

Assess the current estate (existing OpenAI resources, public endpoints, keys in use, quota) → configure the Bicep for a private, keyless, quota-bounded platform → pilot by deploying to a non-production Foundry project with a low TPM quota → validate isolation, keyless access, the content filter, quota, diagnostics, and the budget → expand to production behind pipeline approval, then onboard more projects/models. *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****,* `what-if`*-first and approval-gated, with deployment-stack rollback.*

2. Requirements — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

The category-wide prerequisites (an Azure subscription with AI-service access and rights, Entra identity + managed identity + RBAC, private-networking capability, IaC + Python tooling and source control, a CI/CD platform, data source/classification for grounded solutions, content-safety/responsible-AI posture, and cost controls/quotas) are covered once in **Global Requirements — AI** and are **not repeated here**. Confirm those first. Below are the prerequisites **specific to a secure Azure AI Foundry + Azure OpenAI deployment**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Azure OpenAI (Foundry) resource access approved

Description: Your subscription is approved/enabled to create **Azure OpenAI** (and AI Foundry) resources, and the target **region** offers the models you intend to deploy. **Why required:** Azure OpenAI has historically been an access-managed service and model availability differs by region; you cannot deploy a model that isn't offered where you're deploying. **How to verify:** In the portal, try **Create a resource** → **Azure OpenAI** in the target region, or run `az cognitiveservices account list-skus --kind OpenAI --location <region>`. Check the model/region availability list for your intended models. *Access requirements, model list, and regional availability change — verify current availability in your subscription and region.* **How to implement if missing:** Request access to the AI service if your tenant still requires it; if a model isn't in your region, pick a supported region for that model (you can deploy resources in more than one region).

R2. An existing VNet, subnet, and Private DNS capability for the AI resources

Description: A virtual network with a subnet to hold the **private endpoints**, and the ability to create/link the **private DNS zone(s)** used for private-endpoint resolution (for example `privatelink.openai.azure.com` — *verify the exact zone name(s) for OpenAI / AI Services in your cloud*). **Why required:** Disabling public access without a working private endpoint + DNS makes the resource unreachable; the private path is how apps still call it. **How to verify:** The discovery script lists VNets/subnets and existing private DNS zones; confirm the subnet has room and that you can create/link the zone. Confirm `privateEndpointNetworkPolicies` on the subnet won't block the endpoint. **How to implement if missing:** Provision (or reuse) a subnet for private endpoints and pre-create/link the private DNS zone(s). If your organization uses a central hub for private DNS, plan the zone links there (see the **AI Landing Zone** solution). *Encode ranges/zone names as parameters, never hard-coded literals.*

R3. Custom subdomain enabled on the Azure OpenAI resource

Description: The Azure OpenAI resource is created **with a custom subdomain** (`customSubDomainName`), not the shared regional endpoint. **Why required: Entra ID (keyless/token) authentication and private endpoints require a custom subdomain.** A resource created without one cannot use managed-identity auth and cannot be fronted by a private endpoint the same way. **How to verify:** The resource's endpoint is `https://<your-subdomain>.openai.azure.com/` (not a shared `*.api.cognitive.microsoft.com` regional URL). The Bicep sets `properties.customSubDomainName`. **How to implement if missing:** Set `customSubDomainName` at **create time** in the template. It generally **cannot be added after creation** — if an existing resource lacks one, plan to recreate it (the discovery script flags resources without a custom subdomain).

R4. Data-plane RBAC roles available and assignable to a managed identity

Description: The data-plane roles the platform uses — at minimum **Cognitive Services OpenAI User** (call models) and **Cognitive Services OpenAI Contributor** (also manage deployments) — exist and you can assign them to a **user-assigned managed identity**. **Why required:** With keys disabled (`disableLocalAuth`), these roles are the *only* way an app or script authenticates; without the assignment every call returns 401/403. **How to verify:** `az role definition list --name "Cognitive Services OpenAI User"` returns the role; confirm you have **User Access Administrator/Owner** to assign it. The validation script checks the assignment. **How to implement if missing:** Ensure your deploying identity can assign roles at the resource/RG scope; create the user-assigned managed identity ahead of the app that will use it.

R5. Region quota (TPM) headroom for your intended model deployments

Description: Sufficient **tokens-per-minute (TPM) quota** in the target region for the model(s) and deployment size(s) you plan. **Why required:** A model deployment consumes regional quota; if the region is exhausted the deployment fails, and if quota is too low the app is rate-limited under load. **How to verify:** Check **Quotas** for Azure OpenAI in the target region (portal, or the management SDK the model script uses). *Quota defaults and limits vary by model, region, and subscription type — verify current availability.* **How to implement if missing:** Request a quota increase for the model/region before broad rollout; pilot with a **small** TPM and raise it deliberately (this is also your bill-shock control per Global Requirement G8).

R6. Log Analytics workspace for diagnostics (and a cost budget owner)

Description: A **Log Analytics** workspace to receive the AI resource's diagnostic logs/metrics, and an owner/scope for a **cost budget** with alert recipients. **Why required:** Without a diagnostic destination the platform is unobserved; without a budget an overrun is silent until the invoice (Global Requirement G8). **How to verify:** The workspace exists (or the Bicep will create one) and you can create a diagnostic setting against it; a budget can be created at the resource-group/subscription scope with an action group / email recipients. **How to implement if missing:** Create/choose a Log Analytics workspace and define the budget amount and alert thresholds as parameters. *The budget amount and alert thresholds are the customer's business decision.*

R7. Content-filter (Responsible AI) position for the deployments

Description: A decision on the **content-filter severity thresholds** (and any preview protections such as prompt-shield/jailbreak or protected-material detection) to attach to each model deployment. **Why required:** Generation can emit harmful or ungrounded output; the content filter is the guardrail in front of it,

and its strictness is a deliberate choice (Global Requirement G7). **How to verify:** A RAI/content-filter policy name and thresholds are agreed; the platform attaches it to each deployment. *Some content-safety features are in preview and some require the resource/region to support them — verify current availability.* **How to implement if missing:** Adopt an acceptable-use/RAI position and set the thresholds; the default filter applies until you configure a stricter custom policy (see the **Azure AI Content Safety** solution). *The acceptable-content determination is the customer's legal / trust-and-safety decision, not this guide's.*

R8. Python 3 with current Azure AI SDKs, and az/Az for the helpers

Description: Python 3.x with the Azure SDKs the scripts use — `azure-identity`, `azure-mgmt-cognitiveservices`, `azure-monitor-query`, and the `openai` package configured for Azure — plus the Azure CLI (`az`, `az bicep`) and Az PowerShell for the helpers. **Why required:** The validation and smoke-test scripts authenticate **keylessly** via `DefaultAzureCredential` and exercise the platform through the SDK; the infra is Bicep. **How to verify:** `python --version`, `pip show azure-identity azure-mgmt-cognitiveservices openai`, `az version`, `az bicep version`. *The AI SDKs move fast — verify/pin current versions.* **How to implement if missing:** `pip install azure-identity azure-mgmt-cognitiveservices azure-monitor-query openai` into a pinned virtual environment; install/upgrade the Azure CLI and `az bicep`; commit the pinned versions to the repo.

Global Requirements — AI (category-level)

These requirements apply to every solution in the AI category (building and operating custom AI on Azure — Azure OpenAI / AI Foundry, RAG, Azure ML / MLOps, AI landing zones, content safety). Published once at the category level so listings stay small. Confirm before deploying any AI solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Custom AI touches sensitive data and can emit harmful or ungrounded output. Deploy private-by-default (no public endpoints, keyless auth), ground and permission-trim on your own data, put safety guardrails in front of generation, and pilot in non-production before production — content-acceptability and AI-use-policy decisions belong to the customer's legal / trust-and-safety / compliance functions, not this guide.*

G1. Azure subscription with AI service access and rights

Description: An Azure subscription with access to the required AI services (Azure OpenAI / AI Foundry, Azure AI Search, Azure Machine Learning, Azure AI Content Safety) and Contributor/Owner at the deployment scope. Why: These solutions create and configure real, billable Azure AI resources. Verify: `az account show` succeeds; the AI resource providers are registered and the region is available. *Azure OpenAI/model access and regions vary — verify current availability in your subscription.* Implement: Obtain the subscription/role; request access to the gated AI services where required; pick a supported region.

G2. Entra identity, managed identity, and RBAC

Description: Entra ID identities plus managed identities for keyless service-to-service auth. Why: Keys leak; AI services should authenticate via managed identity + RBAC (least privilege). Verify: Managed

identities can be assigned and the needed data-plane roles exist (e.g. Cognitive Services OpenAI User, Search Index Data Reader). Implement: Enable managed identity on the compute/app; assign least-privilege data-plane roles; store any unavoidable secret in Key Vault.

G3. Private networking capability

Description: A VNet with the ability to create private endpoints and private DNS for the AI services. Why: Public endpoints are a data-exfiltration surface; enterprise AI is private-by-default. Verify: The VNet/subnets and Private DNS zones exist or can be created; public access can be disabled. Implement: Provision the VNet, subnets, and Private DNS; plan private endpoints per service (see the AI landing-zone solution).

G4. IaC + Python tooling and source control

Description: Terraform/Bicep, the Azure CLI, Python 3.x with the relevant Azure AI SDKs, and Git. Why: AI platforms are delivered as versioned code (infra + pipelines + app), so they are reproducible and reviewable. Verify: `az version`, `terraform version`/`az bicep version`, `python --version`, and `git --version` return supported builds. *Verify current SDK/provider versions — the AI SDKs move fast.* Implement: Install and pin the toolchain; store all config in Git.

G5. CI/CD platform

Description: A pipeline (GitHub Actions / Azure DevOps) with OIDC/workload-identity federation to Azure. Why: Infra, model, and app deployments must run through an auditable, approval-gated pipeline — not a laptop. Verify: The pipeline can authenticate to Azure via OIDC and run a plan/deploy step. Implement: Configure the service connection with workload-identity federation; scope its rights.

G6. Data source and classification (for data-grounded solutions)

Description: The private data to ground on (documents, databases) with a known permission/ACL model, and a sensitivity-classification position. Why: RAG must respect source permissions (permission-trimming) and must not expose sensitive data through AI. Verify: The data source, its access model, and applicable sensitivity labels are known. Run the solution's discovery script. Implement: Inventory the data and its ACLs; align sensitivity labeling (see the Purview solutions). *Data-classification determinations are the compliance team's decision.*

G7. Content-safety and responsible-AI posture

Description: An organizational position on acceptable AI content, safety thresholds, and responsible-AI review. Why: Generative AI can emit harmful/ungrounded output and be jailbroken; the organization must define guardrails and review. Verify: A responsible-AI / acceptable-use policy exists; content-safety is enabled where generation occurs. Implement: Adopt an RAI/acceptable-use policy and enable content safety (see the content-safety solution). *Acceptable-content and RAI determinations are the customer's legal / trust-and-safety decision.*

G8. Cost controls and quotas

Description: Budgets, alerts, and per-service quotas (e.g. Azure OpenAI TPM) appropriate to the workload. Why: AI services bill on tokens/compute; unmanaged usage causes bill shock. Verify: Budgets/alerts exist and quotas are set for the deployed services. Implement: Set budgets, cost alerts, and quotas/rate limits before broad rollout. *Spend limits are the customer's business decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

Follow these steps in order. Everything is **pilot-first**: you deploy a **non-production** Foundry project and Azure OpenAI resource with a **low TPM quota**, validate isolation and keyless access, then expand to production behind pipeline approval. Every object you create is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, and the scripts. Placeholder tenant is `contoso.com` / `contoso.onmicrosoft.com`.

> *Run every deployment with `what-if` **first**. Never run an unreviewed apply against production. Examples use the Azure CLI; the CI/CD pipeline runs the same commands behind an approval gate.*

Naming you will create (reuse these exact names)

Object	Name (display)	Resource name (example)
Resource group (non-prod)	`AI Platform RG – Enterprise – NonProd`	`rg-aiplatform-nonprod`
AI Foundry hub	`AI Foundry Hub – Enterprise – NonProd`	`hub-ai-ent-nonprod`
AI Foundry project	`AI Foundry Project – TeamA – NonProd`	`proj-teama-nonprod`
Azure OpenAI resource	`Azure OpenAI – Enterprise – NonProd`	`aoai-ent-nonprod` (subdomain `aoai-ent-nonprod`)
Model deployment	`gpt-4o Deployment – Chat – NonProd`	`gpt-4o-chat`
Content-filter (RAI) policy	`Content Filter – Standard – NonProd`	`rai-standard-nonprod`
Private endpoint (OpenAI)	`Private Endpoint – OpenAI – NonProd`	`pe-aoai-nonprod`
User-assigned managed identity	`Managed Identity – AI App – NonProd`	`id-aiapp-nonprod`
Key Vault	`Key Vault – AI Platform – NonProd`	`kv-aiplat-nonprod`
Log Analytics workspace	`Log Analytics – AI Platform – NonProd`	`law-aiplat-nonprod`

Object	Name (display)	Resource name (example)
Diagnostic setting	`Diagnostics – OpenAI – NonProd`	`diag-aoai-nonprod`
Budget	`Budget – AI Platform – NonProd`	`budget-aiplat-nonprod`
Data-plane role assignment	`OpenAI User – AI App – NonProd`	(role: Cognitive Services OpenAI User)
Deployment stack (non-prod)	`aifoundry-stack – Enterprise – NonProd`	`aifoundry-stack-nonprod`
Deployment stack (prod)	`aifoundry-stack – Enterprise – Prod`	`aifoundry-stack-prod`
CI/CD pipeline	`AI Platform Pipeline – Enterprise – CICD`	(Azure DevOps / GitHub)

Step 1 — Assess: inventory the current AI estate (read-only)

1. Sign in: `az login` (add `--tenant contoso.onmicrosoft.com` if you have several) and `Connect-AzAccount` for the Az modules.
2. Set the working subscription: `az account set --subscription "<non-prod sub id>"`.
3. Run the discovery script from the `scripts/` folder:

`powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Discovery.ps1 -SubscriptionId "<non-prod sub id>"` It reports existing **Cognitive Services / OpenAI** resources and, for each: whether **public network access is Enabled** (a risk), whether **local auth (keys) is enabled**, whether a **custom subdomain** is set (**R3**), model deployments and their TPM, private endpoints, diagnostic settings, and it lists VNets/subnets and existing **private DNS zones** (**R2**). Save the output — it is your baseline and your "what's insecure today" list.

1. Confirm the rights, VNet/subnet, and quota from **R1–R5**.

Step 2 — Configure: scaffold the Bicep repo

1. Clone the empty repo (Global Requirements G4) and add the provided files:

• `AIFOUNDRY-main.bicep` — the platform template (entry point).

• `AIFOUNDRY-Discovery.ps1`, `AIFOUNDRY-Report.ps1` under `scripts/`.

• `AIFOUNDRY-Validate.py`, `AIFOUNDRY-ModelSmokeTest.py` under `scripts/`, plus a pinned `requirements.txt` (**R8**).

1. Create a parameters file per environment (`main.nonprod.bicepparam`, `main.prod.bicepparam`) holding: the **subdomain**, target **VNet/subnet** IDs, **private DNS zone** ID(s), the model + version + **TPM** for the deployment, the **content-filter policy** name/thresholds, the Log Analytics workspace, and the **budget** amount/thresholds (**R2/R5/R6/R7**). Never hard-code these in the template.

2. Commit to a **feature branch** and open a pull request — infrastructure is reviewed, never pushed straight to `main`.

Step 3 — Configure: deploy the private, keyless Azure OpenAI resource (non-prod)

1. Preview with `what-if` at resource-group scope (the Bicep creates the RG-scoped resources; run against the subscription with the RG target):

```
az deployment sub what-if --location eastus --template-file
AIFOUNDRY-main.bicep --parameters main.nonprod.bicepparam
```

1. **Read the preview.** Confirm it creates `Azure OpenAI – Enterprise – NonProd` with `publicNetworkAccess: Disabled`, `disableLocalAuth: true`, and `customSubDomainName` set (**R3**), plus the `Managed Identity – AI App – NonProd`, `Key Vault – AI Platform – NonProd`, and `Log Analytics – AI Platform – NonProd`.

2. Deploy as a deployment stack (rollback-capable):

```
az stack sub create --name "aifoundry-stack-nonprod" --location eastus
--template-file AIFOUNDRY-main.bicep --parameters main.nonprod.bicepparam
--deny-settings-mode denyDelete --action-on-unmanage detachAll
```
Verify deployment-stacks availability in your cloud (R-note); if unavailable, fall back to `az deployment sub create` and note that rollback is manual.

1. Verify in the portal: the resource exists and **Networking** shows **Public network access: Disabled**.

Step 4 — Configure: wire the private endpoint and DNS

1. `what-if` then deploy the private-endpoint block (it is part of `AIFOUNDRY-main.bicep`, toggled by the `deployPrivateEndpoint` parameter): it creates `Private Endpoint – OpenAI – NonProd` on your **R2** subnet with `groupId account`, and the **private DNS zone group** linking it to `privatelink.openai.azure.com` (*verify the exact zone name for your cloud*).

2. Confirm the endpoint's connection state is **Approved** and a private DNS **A record** now exists for `aoai-ent-nonprod.openai.azure.com` pointing to the private IP (**Private endpoint** → **DNS configuration**).

3. From a VM/host **inside the VNet**, `nslookup aoai-ent-nonprod.openai.azure.com` must resolve to the **private** IP. From outside the VNet it should not resolve to a usable public endpoint — that is the isolation working.

Step 5 — Configure: assign least-privilege data-plane RBAC to the managed identity

1. The Bicep creates `OpenAI User – AI App – NonProd` — an assignment of the **Cognitive Services OpenAI User** role to `Managed Identity – AI App – NonProd` at the Azure OpenAI resource scope (**R4**). Confirm it appears under the resource's **Access control (IAM)** → **Role assignments**.

2. Do **not** grant Contributor to app identities; use **Contributor** only for the identity/pipeline that manages deployments, and **User** for identities that only call models. Least privilege by default.

3. Because keys are disabled (`disableLocalAuth`), this assignment is the *only* way to authenticate — there is no key fallback, by design.

Step 6 — Configure: deploy the model with an explicit quota and content filter

1. `what-if` then deploy the model block: it creates the deployment `gpt-4o Deployment – Chat – NonProd` (deployment name `gpt-4o-chat`) of a specific model **version**, with an explicit `capacity` (TPM, in thousands) from your **R5** quota, and attaches the content-filter policy `Content Filter – Standard – NonProd` (**R7**). *Model name/version, region availability, and the exact content-filter API are subject to change — verify current availability.*

2. Keep the pilot **TPM small** (e.g. the minimum that lets you test). It is both your load and your bill-shock ceiling — raise it deliberately later.

3. Confirm the deployment and its attached content filter in the portal (**Azure AI Foundry / OpenAI** → **Deployments**).

Step 7 — Configure: turn on diagnostics and the budget

1. The Bicep creates the diagnostic setting `Diagnostics – OpenAI – NonProd` sending audit/request/response metadata and metrics to `Log Analytics – AI Platform – NonProd`, and the `Budget – AI Platform – NonProd` with the alert thresholds from your parameters (**R6**).

2. Confirm logs are arriving: after a test call (Step 8), run a `AzureDiagnostics`/resource-log query in Log Analytics and see the request appear.

3. Confirm the budget exists with recipients under **Cost Management** → **Budgets**.

Step 8 — Pilot: validate keyless access and isolation end to end

1. Run the Python posture validation (keyless, read-only):

`python AIFOUNDRY-Validate.py --subscription <non-prod sub id> --resource-group rg-aiplatform-nonprod --account aoai-ent-nonprod` It confirms **public access Disabled**, **local auth Disabled**, **custom subdomain set**, the **private endpoint** present, the **role assignment** present, the **deployment + content filter** present, and a **diagnostic setting** configured — the Testing guide's gates. It authenticates via `DefaultAzureCredential` with **no key**.

1. Run the keyless smoke test — it proves an app can actually call the model with a token and **no key**:

`python AIFOUNDRY-ModelSmokeTest.py --endpoint https://aoai-ent-nonprod.openai.azure.com/ --deployment gpt-4o-chat` Run it **from inside the VNet** (a jump host / the CI agent on the VNet). A single, low-token completion returns → keyless + private path work end to end. From outside the VNet the network call should fail — that is isolation confirmed.

1. Generate the posture/usage report: `powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Report.ps1 -SubscriptionId "<non-prod sub id>" -ResourceGroup`

`rg-aiplatform-nonprod`.

Step 9 — Wire the CI/CD pipeline (what-if → approval → deploy)

1. Create the pipeline `AI Platform Pipeline – Enterprise – CICD` in Azure DevOps or GitHub Actions from your repo. Its stages are **Validate** (build Bicep, run `AIFOUNDRY-Validate.py` against non-prod) → **What-If** (posts the preview to the PR) → **Approval** (manual gate) → **Deploy** (deployment stack).

2. Confirm the **service connection** uses **workload identity federation (OIDC)** — no stored secret (Global Requirement G5) — and the **production environment** has the approval gate and reviewer group.

3. Merging to `main` triggers Validate + What-If; the Deploy stage runs only after approval.

Step 10 — Expand: deploy to production, then onboard more projects/models

1. With `main.prod.bicepparam` (production subdomain, VNet, a **higher, deliberate TPM**, stricter content filter if agreed), let the pipeline run **Validate** → **What-If** against production. Review the what-if in the PR.

2. Approve the gate. The pipeline deploys `aifoundry-stack-prod`. Re-run `AIFOUNDRY-Validate.py` against production — all gates must pass before any app points at it.

3. **Onboard a new team** by adding a Foundry **project** under the hub and (if needed) a new **model deployment** with its own TPM and the agreed content filter — parameters only, same template. Each new app gets its **own managed identity** with the **User** role. That is the repeatable payoff.

4. **Rollback** at any point: `az stack sub delete/update` on the named stack removes or reverts exactly what it deployed (use `--action-on-unmanage detachAll` to leave resources in place while detaching stack management).

*Raising a deployment's TPM, swapping a model version, or tightening the content filter are all **parameter changes** through the same `what-if` → approval → deploy path — never a portal click on production.*

5. Testing — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

Validate the platform against a **non-production** Foundry project and Azure OpenAI resource before you expand to production. Each test has an explicit **pass/fail gate**. Reference the scripts by name; they authenticate **keylessly** (`DefaultAzureCredential`) and are read-only except where noted.

Test 1 — Public network access is disabled

• **How:** `python AIFOUNDRY-Validate.py --account aoai-ent-nonprod ...`, or **Azure OpenAI** → **Networking** in the portal.

• **Pass:** `publicNetworkAccess` is **Disabled**; the resource shows no usable public endpoint.

• **Fail:** Still **Enabled** → the private endpoint/DNS is not yet in place (Step 4) or the parameter was not set; do **not** flip it to Disabled until the private path works (Test 3), or you lock yourself out.

Test 2 — Keys are disabled (keyless-only)

• **How:** `AIFOUNDRY-Validate.py` reports `disableLocalAuth`; or try a **key** call and confirm it is rejected.

• **Pass:** `disableLocalAuth` is **true**; an attempted key-based call returns **401** (`AuthenticationTypeDisabled` or similar), while a token call succeeds.

• **Fail:** A key call succeeds → local auth is still enabled; set `disableLocalAuth: true` and redeploy. Both fail → check the role assignment (Test 5) before disabling keys.

Test 3 — Private endpoint resolves and the private path works

• **How:** From a host **inside the VNet**, `nslookup aoai-ent-nonprod.openai.azure.com`; then run `AIFOUNDRY-ModelSmokeTest.py` from inside the VNet.

• **Pass:** The hostname resolves to the **private IP**; the smoke test returns a completion. From **outside** the VNet the call fails to reach the resource.

• **Fail:** Resolves to a public IP / fails inside the VNet → the private DNS zone isn't linked to this VNet or the endpoint isn't Approved; re-check Step 4 (`Private Endpoint – OpenAI – NonProd`).

Test 4 — Custom subdomain is set

• **How:** `AIFOUNDRY-Validate.py` reports the endpoint; or **Overview** → **Endpoint** in the portal.

• **Pass:** Endpoint is `https://aoai-ent-nonprod.openai.azure.com/` (a custom subdomain), which is what makes keyless + private endpoint work (**R3**).

• **Fail:** A shared regional endpoint → the resource was created without a custom subdomain; it must be **recreated** with `customSubDomainName` (cannot be added after creation).

Test 5 — Least-privilege data-plane RBAC is assigned

• **How:** `AIFOUNDRY-Validate.py` reports role assignments on the resource; or **Access control (IAM)** → **Role assignments**.

• **Pass:** `OpenAI User – AI App – NonProd` assigns **Cognitive Services OpenAI User** to `Managed Identity – AI App – NonProd` at the resource scope; app identities have **User**, not Contributor/Owner.

• **Fail:** Missing → the smoke test returns 403; assign the role (Step 5). Over-privileged (Contributor/Owner on an app identity) → tighten to User.

Test 6 — Model deployment has an explicit quota and a content filter

• **How:** `AIFOUNDRY-Validate.py` / `AIFOUNDRY-Report.ps1` list deployments, their **TPM capacity**, and the attached content-filter policy; or **Deployments** in the portal.

• **Pass:** `gpt-4o Deployment – Chat – NonProd` exists with a **non-zero, bounded TPM** and the content-filter policy `Content Filter – Standard – NonProd` attached.

• **Fail:** No TPM ceiling / no filter attached → the deployment is unbounded or unguarded; redeploy the model block (Step 6). *If a model/version is unavailable in the region, verify current availability (R1).*

Test 7 — The content filter actually blocks

• **How:** Send a deliberately disallowed prompt via the smoke test's `--filter-check` mode (safe, canned test prompt) and observe the response.

• **Pass:** The request is **filtered** (content-filter error / finish reason `content_filter`), proving the guardrail is live at the configured thresholds.

• **Fail:** Disallowed content passes → the deployment has no policy attached or thresholds are permissive; re-check Step 6 and **R7**. *Acceptable thresholds are your trust-and-safety decision.*

Test 8 — Diagnostics flow and the budget exists

• **How:** After a test call, run a resource-log query in `Log Analytics – AI Platform – NonProd`; check **Cost Management** → **Budgets**.

• **Pass:** The test request appears in Log Analytics within a few minutes; `Budget – AI Platform – NonProd` exists with alert recipients.

• **Fail:** No logs → the diagnostic setting isn't pointed at the workspace (Step 7). No budget → create it (Step 7); an AI platform without a spend alarm is a bill-shock risk.

Test 9 — Deployment stack tracks and can roll back

• **How:** `az stack sub show --name aifoundry-stack-nonprod`.

• **Pass:** The stack lists its managed resources; a `what-if` of `az stack ... delete/update` shows exactly what would change.

• **Fail:** Stack not found → you deployed with plain `az deployment` instead of `az stack`; redeploy via the stack so rollback is available.

Test 10 — Pipeline enforces what-if and approval

• **How:** Push a trivial parameter change on a branch; open a PR; watch the pipeline.

• **Pass: Validate** and **What-If** run automatically and post the preview; **Deploy** pauses at the approval gate and proceeds only after sign-off.

• **Fail:** Deploy runs without pausing → the environment approval check isn't attached to the production stage; fix in the pipeline environment configuration.

Exit criteria

All ten tests pass against the non-production resource — public access disabled with a working private path, keyless-only auth, least-privilege RBAC, a quota-bounded and content-filtered deployment, live diagnostics and a budget, stack rollback available, and an approval-gated pipeline — **only then** expand to

production and onboard further projects and models.

6. Troubleshooting — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

Common failures during a secure Azure OpenAI / AI Foundry rollout, as **symptom** → **cause** → **fix**. Re-run `AIFOUNDRY-Validate.py` after any fix to re-check the gates.

Access, resource creation, and region

Symptom: You can't create the Azure OpenAI resource, or the SKU/kind isn't offered. **Cause:** The subscription isn't approved for the AI service, or the resource isn't offered in the region (**R1**). **Fix:** Request access to the AI service if your tenant still gates it; pick a region that offers Azure OpenAI. `az cognitiveservices account list-skus --kind OpenAI --location <region>`. *Verify current access requirements and regional availability.*

Symptom: Model deployment fails with a quota/capacity error. **Cause:** No TPM quota left for that model in the region, or you requested more capacity than granted (**R5**). **Fix:** Lower the deployment `capacity`, request a quota increase for the model/region, or deploy in a region with headroom. Pilot with a small TPM and raise deliberately.

Custom subdomain (the keyless + private-endpoint prerequisite)

Symptom: Managed-identity auth returns 401, or you can't attach a private endpoint the expected way. **Cause:** The resource was created **without a custom subdomain**; the endpoint is a shared regional URL (**R3**). **Fix:** `customSubDomainName` generally **cannot be added after creation** — recreate the resource with it set (the Bicep does this). `AIFOUNDRY-Discovery.ps1` flags resources with no subdomain.

Keyless / RBAC (401 / 403)

Symptom: Your app or `AIFOUNDRY-ModelSmokeTest.py` gets **401 Unauthorized** / **403 PermissionDenied** calling the model. **Cause:** The identity has no data-plane role on the resource; with `disableLocalAuth: true` there is **no key fallback** (**R4**, Test 5). **Fix:** Assign **Cognitive Services OpenAI User** to the app's managed identity at the resource scope (`OpenAI User – AI App – NonProd`). Wait for the assignment to propagate (can take a few minutes), then retry.

Symptom: A key-based call used to work and now returns `AuthenticationTypeDisabled` (or similar). **Cause:** `disableLocalAuth: true` turned keys off — this is intended (Test 2). **Fix:** Switch the caller to token auth (`DefaultAzureCredential` + the `https://cognitiveservices.azure.com/.default` scope), as the smoke-test script shows. Do **not** re-enable local auth.

Symptom: `DefaultAzureCredential` can't get a token locally. **Cause:** You're not logged in, or no managed identity is available in the runtime. **Fix:** `az login` for local runs; ensure the managed identity is assigned to the compute (VM/App Service/AKS/CI agent) for runtime. Confirm the identity holds the OpenAI User role.

Private endpoint / DNS / networking

Symptom: After disabling public access, apps can't reach the resource at all. **Cause:** Public access was disabled **before** a working private endpoint + DNS existed (Test 1 warning), or the caller is outside the VNet. **Fix:** Ensure `Private Endpoint – OpenAI – NonProd` is **Approved** and the private DNS zone is linked to the caller's VNet; run apps/scripts **inside** the VNet. Order matters: prove the private path, *then* disable public access.

Symptom: `nslookup <account>.openai.azure.com` returns a public IP from inside the VNet. **Cause:** The private DNS zone (`privatelink.openai.azure.com` — *verify the exact zone name for your cloud*) isn't linked to that VNet, or the A record wasn't created by the DNS zone group. **Fix:** Confirm the zone name, that the VNet has a **virtual network link** to the zone, and that the endpoint's **DNS configuration** created the A record; re-check with `AIFOUNDRY-Discovery.ps1`.

Symptom: The private endpoint can't be created on the subnet. **Cause:** `privateEndpointNetworkPolicies` blocks it, or the subnet is full (**R2**). **Fix:** Adjust the subnet's private-endpoint network policy and ensure address space; the discovery script reports both per subnet.

Model deployment / content filter

Symptom: The model/version in the Bicep no longer deploys. **Cause:** The model or version was retired or isn't offered in the region — model availability changes. **Fix:** *Verify current model + version availability in your region* and update `modelName`/`modelVersion` parameters; re-`what-if` before deploy.

Symptom: A clearly disallowed prompt is **not** filtered (Test 7 fail). **Cause:** No content-filter policy is attached to the deployment, or thresholds are permissive (**R7**). **Fix:** Attach `Content Filter – Standard – NonProd` to the deployment and set thresholds to your trust-and-safety decision; redeploy the model block. *Some filter features are in preview — verify availability.*

Symptom: Legitimate prompts are being over-blocked after tightening the filter. **Cause:** Thresholds set too strict for the workload. **Fix:** This is a trust-and-safety tuning decision, not a bug — adjust thresholds deliberately and re-test. Change it as a parameter through the pipeline, not a portal click on prod.

Monitoring / cost

Symptom: No requests appear in Log Analytics after test calls. **Cause:** The diagnostic setting isn't pointed at the workspace, or the wrong log category is enabled (Test 8). **Fix:** Confirm `Diagnostics – OpenAI – NonProd` targets `Log Analytics – AI Platform – NonProd` with the request/audit categories; wait a few minutes for ingestion. *Verify current diagnostic category names.*

Symptom: Spend climbed with no warning. **Cause:** No budget/alert, or TPM set too high (**R5/R6**). **Fix:** Create `Budget – AI Platform – NonProd` with alert recipients; cap each deployment's TPM to a deliberate ceiling. `AIFOUNDRY-Report.ps1` shows budget status and per-deployment TPM.

Deployment stacks / IaC

Symptom: `az stack sub show` reports the stack is missing after you deployed. **Cause:** You deployed with `az deployment sub create` instead of `az stack sub create`, so there's no rollback unit (Test 9). **Fix:** Redeploy via `az stack sub create` with the named stack so resources are tracked and rollback is available.

Symptom: `what-if` shows unexpected **Delete/Modify** on the resource. **Cause:** A parameter changed the resource name (ARM sees a replace), or a stack-managed resource was edited in the portal (drift).

Fix: Stop — do not approve. Diff the parameter files, run `AIFOUNDRY-Report.ps1` for drift, reconcile, then re-`what-if` until clean.

Symptom: `az stack` / `az bicep` is "not a recognized command," or the Foundry resource types don't resolve. **Cause:** Azure CLI / Bicep too old, or the Foundry hub/project resource model differs from the template (**R8**, Overview note). **Fix:** `az bicep upgrade` and update the CLI; *verify the current AI Foundry resource types and apiVersions for your tenant/cloud* and adjust the template. Pin versions in the repo.

Pipeline

Symptom: The Deploy stage runs without pausing for approval. **Cause:** The environment approval check isn't attached to the production stage (Step 9). **Fix:** Add required reviewers to the production environment and confirm the deploy job references it; re-run (Test 10).

Symptom: Pipeline auth fails with a token/credential error. **Cause:** The service connection isn't configured for workload identity federation, or its subject/scope is wrong (**G5**). **Fix:** Reconfigure the service connection for OIDC/workload identity federation, scoped to the deployment target; never fall back to a stored secret.

7. Best Practices — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

Keep it clean

• **Private-by-default, always.** Create the resource with `publicNetworkAccess: Disabled` and a private endpoint from the first deploy — don't stand it up public "just to test" and plan to close it later. The window between the two is a real exposure, and the private path is easy to prove before you disable public access.

• **Keyless, no exceptions.** Set `disableLocalAuth: true` and authenticate every app and script with a **managed identity** and the least-privilege data-plane role. There is no such thing as a "temporary" key — a key is a portable secret that outlives the developer who created it. Regenerating keys is not a control; disabling them is.

• **Least privilege on the data plane.** App identities that only call models get **Cognitive Services OpenAI User**; only the identity/pipeline that *manages* deployments gets **Contributor**. Never grant Owner to a workload. One managed identity per app, scoped to the one resource it needs.

• **Custom subdomain at create time.** It's required for keyless + private endpoints and generally can't be added later — set `customSubDomainName` in the template so you never have to recreate a resource to fix it.

• **Quota is your bill-shock control.** Give every deployment an **explicit, deliberate TPM** and raise it consciously. An unbounded or over-large deployment is how a runaway loop or a launch spike becomes a five-figure invoice. Pilot small.

• **A content filter on every deployment.** Attach a RAI/content-filter policy to each deployment and set the thresholds deliberately — never ship a deployment with generation exposed and no guardrail. *Verify preview status of prompt-shield/protected-material features before relying on them.*

• **Everything through the pipeline.** After the bootstrap, never `az deployment`/`az stack` from a laptop against production. The what-if → approval → deploy path is the only production route, so every change — a new model, a TPM bump, a stricter filter — is reviewed and recorded.

• **What-if is a gate, not a formality.** Read every preview. An unexpected **delete** or **modify** on a resource you didn't intend to touch means **stop** and investigate before you approve.

• **Parameterize, never hard-code.** Subdomains, VNet/subnet and DNS-zone IDs, model/version, TPM, filter thresholds, workspace, and budget live in `.bicepparam` files — one per environment. The same `AIFOUNDRY-main.bicep` deploys non-prod and prod; only parameters differ.

• **Use deployment stacks for rollback.** Deploy as stacks with `denyDelete` so out-of-band portal edits are blocked, and keep `--action-on-unmanage` explicit so a rollback does exactly what you expect.

• **Observe every call.** Keep the diagnostic setting to Log Analytics on, and keep a budget with alerts. An AI platform you can't see is one you can't govern or cost-control.

• **Pin the toolchain.** Pin Bicep, the Azure CLI, and the Python SDK versions in the repo. The AI SDKs move fast; IaC that "worked last month" breaks on a silent update. *Verify current supported versions periodically.*

Decisions that belong to the customer

• **Acceptable AI content and use** — the content-filter thresholds and what your organization permits are your **legal / trust-and-safety** decision; the platform enforces them, it does not define them (Global Requirement G7).

• **Data classification and grounding data** — what data may be sent to or grounded in the model is your **compliance** decision (Global Requirement G6). This platform is the model layer; grounding on private data is the sibling RAG solution.

• **Spend limits** — the budget amount, TPM ceilings, and alert thresholds are your **business** decision (Global Requirement G8).

• **Production change approval** — who approves production deploys and the soak period are your **change-control** decision; the pipeline provides the gate.

• **Government vs. commercial cloud** — sovereign-cloud requirements (which region/cloud, which DNS zone names, which features are available) are your **compliance** decision; *verify availability in your environment*.

Where to go next

Once the secure platform is live, adjacent catalog areas extend it:

• **AI** — the **Enterprise RAG Platform (Azure AI Search + private data)** solution to ground the model on your own permission-trimmed data; the **Azure AI Content Safety & Responsible-AI Guardrails** solution to deepen the content-filter and add prompt-shield/jailbreak protections; the **MLOps (Azure Machine Learning)** solution for the model-training/registry side; and the **AI Landing Zone** solution for the org-wide governance and hub-spoke backbone this platform deploys *into*.

• **Copilot / Purview AI** — if instead you need to **govern Microsoft 365 Copilot** over your M365 data (a different job than building custom AI), those catalog areas cover Copilot readiness, data-oversharing control, and AI-interaction auditing. Complementary, not overlapping.

• **Security / Identity** — Conditional Access and PIM to harden the identities that manage this platform; **Key Vault** patterns for any unavoidable downstream secret.

• **DevOps** — the **Azure Landing Zone (Bicep/Terraform)** and **Intune-as-Code** solutions to bring the rest of the estate under the same code-and-pipeline discipline.

8. Deliverables Summary — Azure AI Foundry & Azure OpenAI: Secure Enterprise Deployment

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — private, keyless, quota-managed, monitored
1. Solution Overview	What the platform is — AI Foundry hub/project + Azure OpenAI; how private endpoints, keyless managed-identity auth, model deployments/quota, and content filtering work; scope, limits, and the boundary vs. Copilot/Purview-AI governance
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (AI)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered steps — discover, deploy private+keyless Azure OpenAI as code, wire private endpoint/DNS, assign least-privilege RBAC, deploy a quota-bounded and content-filtered model, turn on diagnostics + budget, pilot, then prod
5. Testing	Validation procedures with pass/fail gates
6. Troubleshooting	Field issues → cause → fix
7. Best Practices	Field-proven recommendations and where to go next

Automation artifacts

File	What it does
`AIFOUNDRY-Discovery.ps1`	**PowerShell, read-only.** Estate discovery — every Cognitive Services / OpenAI resource and its risk posture (public access, keys enabled, custom subdomain, model deployments + TPM, private endpoints, diagnostics), plus VNets/subnets and private DNS zones. Your "what's insecure today" baseline.

File	What it does
`AIFOUNDRY-main.bicep`	**Bicep (IaC), the core.** Deploys the platform as code — AI Foundry hub/project + Azure OpenAI resource with `publicNetworkAccess: Disabled`, `disableLocalAuth: true`, and a custom subdomain; a user-assigned managed identity + least-privilege data-plane role assignment; a private endpoint + private DNS zone group; a model deployment with explicit TPM and a content-filter policy; a Key Vault; and a diagnostic setting + budget. Toggled by parameters, `what-if`-first.
`AIFOUNDRY-Validate.py`	**Python (Azure SDK), keyless & read-only.** Posture validation against the Testing gates — confirms public access disabled, local auth disabled, custom subdomain set, private endpoint present, least-privilege role assigned, deployment + content filter present, and diagnostics configured. Authenticates via `DefaultAzureCredential` (no key).
`AIFOUNDRY-Report.ps1`	**PowerShell, read-only.** Posture + usage/cost report — resource security summary, model deployments and TPM, recent request/token volume from Log Analytics, and budget/alert status, for review and drift-spotting.
`AIFOUNDRY-ModelSmokeTest.py`	**Python (openai SDK for Azure), keyless.** Proves an app can call the model with a **managed-identity token and no key** over the private path — a single low-token completion; plus a safe `--filter-check` mode that confirms the content filter blocks a canned disallowed prompt. Run from inside the VNet.

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five automation artifacts (`AIFOUNDRY-main.bicep`, two Python scripts, and two PowerShell scripts) plus a pinned `requirements.txt` in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — `what-if`-first, keyless, quota-bounded, approval-gated, with deployment-stack rollback.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

A secure, governed **Azure OpenAI + AI Foundry** platform expressed as code in Git — private, keyless, quota-managed, and monitored — that your teams build enterprise AI applications on with confidence, proven on a non-production project before it reaches production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AIFOUNDRY-Discovery.ps1

Purpose: Builds your "what's insecure today" baseline before deploying the secure platform. For every Cognitive Services / OpenAI account in the subscription it reports the risk posture: public network access, whether account keys (local auth) are enabled, whether a custom subdomain is set (required for keyless + private endpoints), the model deployments and their TPM capacity, private endpoints, and diagnostic settings. It also lists VNets/subnets and existing private DNS zones so you can plan the private endpoint (R2). Nothing is changed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Discovery.ps1 -SubscriptionId "<sub id>" Requires the Az PowerShell modules (Az.Accounts, Az.CognitiveServices, Az.Network, Az.Monitor). Run Connect-AzAccount first. *Verify current Az module versions.* EXPECTED RESULTS A per-resource posture table (public access / keys / subdomain / deployments / PE / diagnostics) and a networking summary (VNets, subnets, private DNS zones). Read-only — no changes made. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$SubscriptionId, [string]$OutputCsv = "")

How to run: powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Discovery.ps1 -SubscriptionId "<sub id>" Requires the Az PowerShell modules (Az.Accounts, Az.CognitiveServices, Az.Network, Az.Monitor). Run Connect-AzAccount first. *Verify current Az module versions.* EXPECTED RESULTS A per-resource posture table (public access / keys / subdomain / deployments / PE / diagnostics) and a networking summary (VNets, subnets, private DNS zones). Read-only — no changes made. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$SubscriptionId, [string]$OutputCsv = "")

AIFOUNDRY-ModelSmokeTest.py

Purpose: Proves the two hardest-to-fake controls actually work end to end: 1. KEYLESS access -- authenticates with a managed-identity / az-login TOKEN (no account key) using DefaultAzureCredential + the Cognitive Services token scope, and makes ONE small, low-token chat completion against your deployment. 2. CONTENT FILTER (optional --filter-check) -- sends a single CANNED, mild policy-test prompt and confirms the request is blocked/filtered, proving the guardrail is live. Run it FROM INSIDE THE VNET (a jump host or the CI agent on the VNet). Success = keyless + the private path both work. From outside the VNet the network call should fail -- that is isolation. It sends at most one or two tiny requests; it changes no configuration. HOW TO RUN pip install openai azure-identity python AIFOUNDRY-ModelSmokeTest.py --endpoint https://aoai-ent-nonprod.openai.azure.com/ \ deployment gpt-4o-chat [--filter-check] [--api-version 2024-10-21] (az login first, or run as the managed identity that holds Cognitive Services OpenAI User.) Verify the current Azure OpenAI api-version and SDK version for your resource.* EXPECTED RESULTS "KEYLESS CALL OK" plus the (truncated) model reply. With --filter-check, "CONTENT FILTER ACTIVE" when the canned prompt is blocked. Non-zero exit on failure. .NOTES White-label. Keyless (DefaultAzureCredential -- NO key). Minimal, read-only-ish smoke test. Sends no sensitive data; the filter-check prompt is a mild, canned safety probe. """ import argparse import sys

How to run: pip install openai azure-identity python AIFOUNDRY-ModelSmokeTest.py --endpoint https://aoai-ent-nonprod.openai.azure.com/ \ deployment gpt-4o-chat [--filter-check] [--api-version 2024-10-21] (az login first, or run as the managed identity that holds Cognitive Services OpenAI User.)

Verify the current Azure OpenAI api-version and SDK version for your resource.* EXPECTED RESULTS "KEYLESS CALL OK" plus the (truncated) model reply. With --filter-check, "CONTENT FILTER ACTIVE" when the canned prompt is blocked. Non-zero exit on failure. .NOTES White-label. Keyless (DefaultAzureCredential -- NO key). Minimal, read-only-ish smoke test. Sends no sensitive data; the filter-check prompt is a mild, canned safety probe. """ import argparse import sys

AIFOUNDRY-Report.ps1

Purpose: Produces a review-and-drift report for one AI platform resource group: the security posture of each Azure OpenAI resource (public access, keys, subdomain, private endpoints, diagnostics), the model deployments and their TPM ceilings, recent request/token volume pulled from Log Analytics, and the cost budget / alert status. Use it in regular reviews to confirm nothing has drifted away from private-keyless-bounded-monitored. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Report.ps1 -SubscriptionId "<sub id>" ` -ResourceGroup rg-aiplatform-nonprod [-LogAnalyticsWorkspaceId <customerId GUID>] Requires Az.Accounts, Az.CognitiveServices, Az.Monitor, Az.Consumption, and (for usage) Az.OperationalInsights. Run Connect-AzAccount first. *Verify current Az module versions.* EXPECTED RESULTS Printed posture/usage/cost sections. Read-only — no changes are made. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$SubscriptionId, [Parameter(Mandatory = $true)] [string]$ResourceGroup, [string]$LogAnalyticsWorkspaceId = "", [int]$UsageLookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\AIFOUNDRY-Report.ps1 -SubscriptionId "<sub id>" ` -ResourceGroup rg-aiplatform-nonprod [-LogAnalyticsWorkspaceId <customerId GUID>] Requires Az.Accounts, Az.CognitiveServices, Az.Monitor, Az.Consumption, and (for usage) Az.OperationalInsights. Run Connect-AzAccount first. *Verify current Az module versions.* EXPECTED RESULTS Printed posture/usage/cost sections. Read-only — no changes are made. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$SubscriptionId, [Parameter(Mandatory = $true)] [string]$ResourceGroup, [string]$LogAnalyticsWorkspaceId = "", [int]$UsageLookbackHours = 24)

AIFOUNDRY-Validate.py

Purpose: Confirms the platform matches the Testing guide's pass/fail gates: - public network access DISABLED - local auth (account keys) DISABLED -> keyless-only - a custom subdomain is set (required for keyless + private endpoints) - at least one private endpoint connection is present - a least-privilege data-plane role (Cognitive Services OpenAI User) is assigned - at least one model deployment exists WITH a bounded TPM capacity - a diagnostic setting is configured Authenticates via DefaultAzureCredential (your az login / a managed identity) with NO KEY. Read-only: it inspects resource management state; it makes no changes. HOW TO RUN pip install azure-identity azure-mgmt-cognitiveservices azure-mgmt-authorization azure-mgmt-monitor python AIFOUNDRY-Validate.py --subscription <sub id> --resource-group rg-aiplatform-nonprod \ account aoai-ent-nonprod (az login first, or run as a managed identity with Reader on the resource.) Verify current Azure SDK versions -- the AI SDKs move fast.* EXPECTED RESULTS A PASS/FAIL line per gate and a non-zero exit code if any gate fails. No changes are made. .NOTES White-label. Read-only. Keyless (DefaultAzureCredential). Makes no changes. """ import argparse import sys

How to run: pip install azure-identity azure-mgmt-cognitiveservices azure-mgmt-authorization azure-mgmt-monitor python AIFOUNDRY-Validate.py --subscription <sub id> --resource-group rg-aiplatform-nonprod \ account aoai-ent-nonprod (az login first, or run as a managed identity with Reader

on the resource.) Verify current Azure SDK versions -- the AI SDKs move fast.* EXPECTED RESULTS A PASS/FAIL line per gate and a non-zero exit code if any gate fails. No changes are made. .NOTES White-label. Read-only. Keyless (DefaultAzureCredential). Makes no changes. """ import argparse import sys

AIFOUNDRY-main.bicep

Purpose: // Declares the AI platform as code, private-by-default and keyless: // - Azure OpenAI (Cognitive Services, kind=OpenAI) with a CUSTOM SUBDOMAIN, // publicNetworkAccess = Disabled, and disableLocalAuth = true (no keys). // - A user-assigned MANAGED IDENTITY + a least-privilege data-plane role // assignment (Cognitive Services OpenAI User) on the resource. // - A PRIVATE ENDPOINT (groupId 'account') + private DNS zone group. // - A MODEL DEPLOYMENT with an explicit TPM 'capacity' and an attached // content-filter (RAI) policy. // - A Key Vault (for any unavoidable downstream secret), a Log Analytics // DIAGNOSTIC SETTING, and a cost BUDGET with alerts. // Each block is toggled by a parameter so you can what-if and deploy // modules independently. Pilot-first. // // HOW TO RUN (what-if FIRST, always) — subscription-scoped, targets an RG // az deployment sub what-if \ // --location eastus \ // --template-file AIFOUNDRY-main.bicep --parameters main.nonprod.bicepparam // Deploy as a deployment stack (rollback-capable): // az stack sub create --name aifoundry-stack-nonprod --location eastus \ // --template-file AIFOUNDRY-main.bicep --parameters main.nonprod.bicepparam \ // --deny-settings-mode denyDelete --action-on-unmanage detachAll // // EXPECTED RESULTS // A private, keyless Azure OpenAI resource, a managed identity with the // least-privilege role, a private endpoint + DNS, one quota-bounded and // content-filtered model deployment, plus Key Vault, diagnostics and a // budget — created idempotently (re-running converges, does not duplicate). // // White-label. Deploy plan/what-if first, through the reviewed pipeline. // NOTE: apiVersions, the AI Foundry hub/project resource model, model names/ // versions, region availability, the content-filter (raiPolicies) API, and // the private DNS zone name are marked *verify* — confirm current values and // availability in YOUR subscription, region, and cloud (Commercial vs Gov) // before relying on them. // ==========